



Python Functions

Practical Skills Module 3: Topic 3

[Visit our website](#)

Practical Skills Module 3: Topic 3

Topic Code	PM-03-PS03
Topic	Define functions in Python

Scope of Practical Skill

Given a range the learner must be able to:

- PA0301 Define the main function main() in Python
- PA0302 Use the keyword def to declare the function and follow this up with the function name.
- PA0303 Add parameters to the function
- PA0304 Add statements that the functions should execute
- PA0305 End the function with a return statement if the function should return something.
- PA0306 Use Python functions:
 - Lamda (anonymous) function
 - Abs() Function: Absolute Value
 - Round() function
 - Range() function: Float, List, For loop
 - Map() function
 - Timeit()
 - Yield (Generator and Yield vs Return)
 - Queue (FIFO, LIFO)
 - Counter
 - Enumerate() function: (Loop, Tuple, String)
 - Time.sleep()
 - Type()
 - Isinstance()
 - New line
 - Modules
 - Package
 - Global, local and nonlocal variables
 - Global keyword and global variable

Applied Knowledge

- AK0301 Concept, definition and functions of Python programming functionalities

Internal Assessment Criteria

- IAC0301 Expected results with Python functions are achieved

Resources:

Knowledge	Information from Python Programmer Curriculum
National Curriculum Framework	900221-000-00-PM-03



Practical task

Follow the facilitator's instructions to complete the following activities:

Create a Python file named **order_processing.py** and ensure it is stored in your task folder

Scenario: E-Commerce Order Processing System

An e-commerce company requires a Python-based system to manage essential tasks such as calculating discounts, processing customer orders, tracking inventory, managing queues, and generating summary reports. This system should demonstrate the use of functions, built-in tools, and control structures to simulate a functional order processing pipeline.

Step 1: Define the Program Entry Point

Create a `main()` function to serve as the entry point for your application. Within this function, you can organise calls to other components (e.g., process orders, generate reports).

Step 2: Define Functions for Core Features

- Define a function `calculate_discount(price, discount_rate)` to compute the discounted price of a product.

- Implement `process_order(order_id, status)` to simulate basic order handling logic (e.g., changing order status to "processed" or "shipped").

Step 3: Use a Return Statement

Ensure the `process_order()` function returns the final status of the order after processing, so it can be used in logs or reports.

Step 4: Apply Common Python Functions

Demonstrate practical use of several built-in Python functions to add utility to your program:

- Lambda function: Compute total price including tax.
- `abs()`: Calculate the absolute difference between stock = 20 and demand = 30.
- `round()`: Round a price (e.g. 49.56789) to two decimal places.
- `range()`: Iterate through a list of order IDs, e.g. `range(101, 105)`.
- `map()`: Apply a discount to multiple prices: [100, 200, 300].
- `timeit` module: Measure how long a function (e.g. `calculate_discount`) takes to execute.

Step 5: Introduce Advanced Concepts

Use the `Counter` class from the `collections` module to:

- Count how many times each product category appears in a list of products (e.g., electronics, clothing, groceries).

Step 6: Use Iteration and Conditional Logic

Incorporate common iteration and type-checking tools:

- `enumerate()`: Iterate through a list of orders and print both index and content.
- `time.sleep()`: Simulate a delay (e.g., 1 second) between processing orders to reflect real-time action.
- `type()` and `isinstance()`: Confirm the data types of order-related variables.

- Membership testing (in): Check if a specific product or order ID exists in a given list before processing.

Step 7: Demonstrate Variable Scope and Modification

Show the use of variable scopes:

- Create a local variable inside a function and explain its scope.
- Use the global keyword to modify a global variable (e.g. total orders processed).
- Introduce a nonlocal variable inside a nested function to modify a parent function's variable.

At the end of this task, your program should simulate a basic yet structured order processing pipeline that showcases key Python concepts through real-world e-commerce logic.



Take note

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.
